UNIVERSITI MALAYA

NFMN

SIM 1005 / SIN 1002

Fundamentals of Spreadsheets / Introduction to Worksheet

Lecture 10: VBA – Functions, Objects & UserForm

By

Assoc. Prof. Dr Noor Fadiya Mohd Noor

Institute of Mathematical Sciences

Objectives

- To differentiate between Function and Sub in VBA
- To execute commands using Function and Sub.
- To manipulate Excel objects for VBA.
- To create UserForm in VBA.

Original notes by NFM Noor

VBA Continues..

- VBA includes:
  13) Function and Sub
  14) Application Object
  15) ActiveX Controls
  16) Userform

Original notes by NFM Noor

Function and Sub

- A function in Excel VBA can return a value while a sub cannot

Function

If you want Excel VBA to perform a task that returns a result, you can use a function. Place a function into a module (In the Visual Basic Editor, click Insert, Module). For example, the function with name Area.

```
Function Area(x As Double, y As Double) As Double

Area = x * y

End Function
```

Original notes by NFM Noor

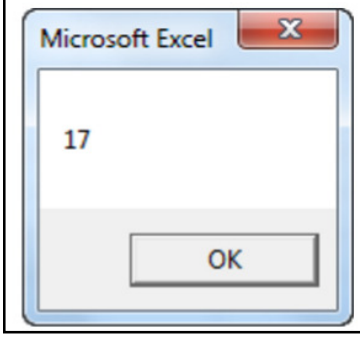
Place a command button on your worksheet and add the following code lines:

```
Dim z As Double

z = Area(3, 5) + 2

MsgBox z
```

Result when you click the command button on the sheet:



Original notes by NFM Noor

Sub

If you want Excel VBA to perform some actions, you can use a sub. Place a sub into a module (In the Visual Basic Editor, click Insert, Module). For example, the sub with name Area.

```
Sub Area(x As Double, y As Double)

MsgBox x * y

End Sub
```

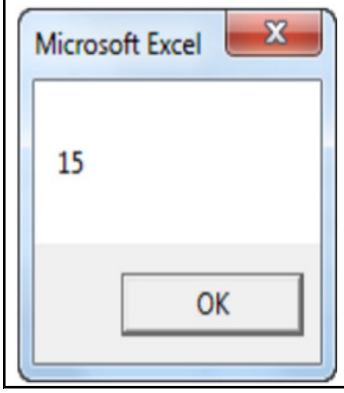
Explanation: This sub has two arguments (of type Double). It does not have a return type! You can refer to this sub (call the sub) from somewhere else in your code by simply using the name of the sub and giving a value for each argument.

Original notes by NFM Noor

Place a command button on your worksheet and add the following code line:

```
Area 3, 5
```

Result when you click the command button on the sheet:



Original notes by NFM Noor

Application Object

- The mother of all Excel objects.
- Gives access to other Excel related options:
  1) WorksheetFunction
  2) ScreenUpdating
  3) DisplayAlerts
  4) Calculation

Original notes by NFM Noor

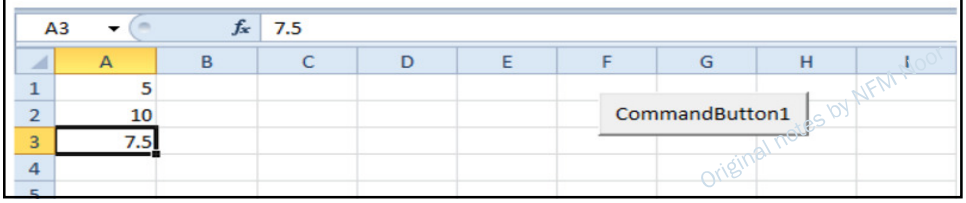
WorksheetFunction

You can use the WorksheetFunction property in Excel VBA to access Excel functions.

1. For example, place a command button on your worksheet and add the following code line:

```
Range("A3").Value = Application.WorksheetFunction.Average(Range("A1:A2"))
```

When you click the command button on the worksheet, Excel VBA calculates the average of the values in cell A1 and cell A2 and places the result into cell A3.



Note: instead of Application.WorksheetFunction.Average, simply use WorksheetFunction.Average. If you look at the formula bar, you can see that the formula itself is not inserted into cell A3. To insert the formula itself into cell A3, use the following code line:

```
Range("A3").Value = "=AVERAGE(A1:A2)"
```

ScreenUpdating

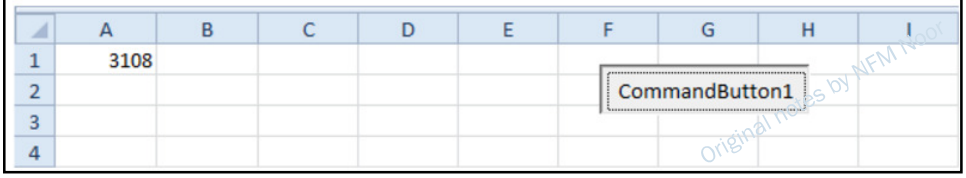
Sometimes you may find it useful to disable screen updating (to avoid flickering) while executing code. As a result, your code will run faster.

1. For example, place a command button on your worksheet and add the following code lines:

```
Dim i As Integer

For i = 1 To 10000
    Range("A1").Value = i
Next i
```

When you click the command button on the worksheet, Excel VBA displays each value a tiny fraction of a second and this can take some time.



Original notes by NFM Noor

2. To speed up the process, update the code as follows.

```
Dim i As Integer

Application.ScreenUpdating = False

For i = 1 To 10000
    Range("A1").Value = i
Next i

Application.ScreenUpdating = True
```

As a result, your code will run much faster and you will only see the end result (10000).

Original notes by NFM Noor

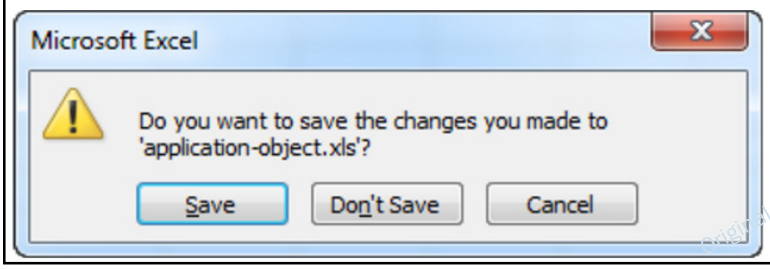
DisplayAlerts

You can instruct Excel VBA not to display alerts while executing code.

1. For example, place a command button on your worksheet and add the following code line:

```
ActiveWorkbook.Close
```

When you click the command button on the worksheet, Excel VBA closes your Excel file and asks you to save the changes you made.



Original notes by NFM Noor

2. To instruct Excel VBA not to display this alert while executing code, update the code as follows.

```
Application.DisplayAlerts = False

ActiveWorkbook.Close

Application.DisplayAlerts = True
```

As a result, Excel VBA closes your Excel file, without asking you to save the changes you made. Any changes are lost.

Original notes by NFM Noor

Calculation

By default, calculation is set to automatic. As a result, Excel recalculates the workbook automatically each time a value affecting a formula changes. If your workbook contains many complex formulas, you can speed up your macro by setting calculation to manual.

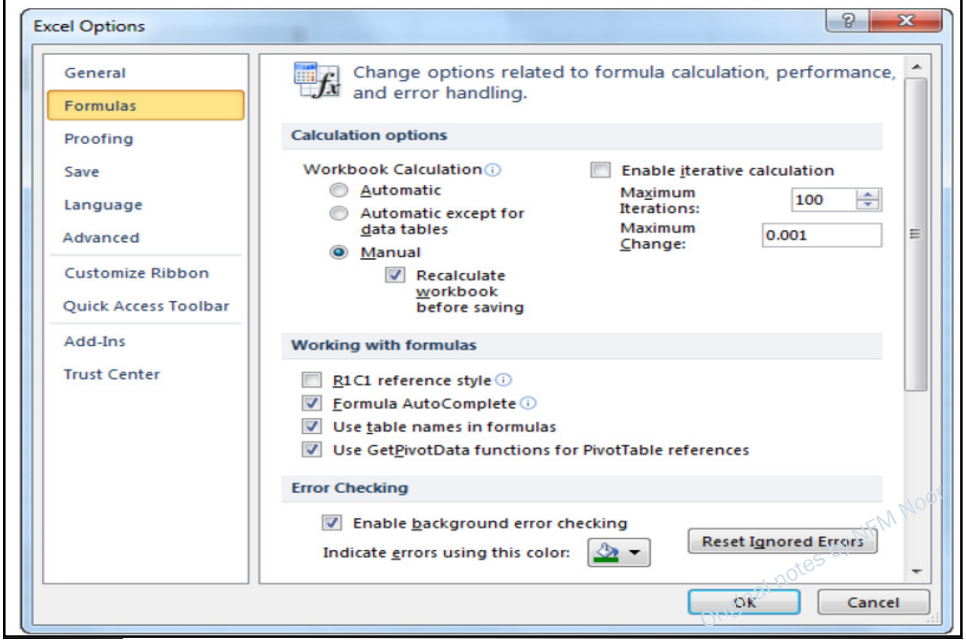
1. For example, place a command button on your worksheet and add the following code line:

```
Application.Calculation = xlCalculationManual
```

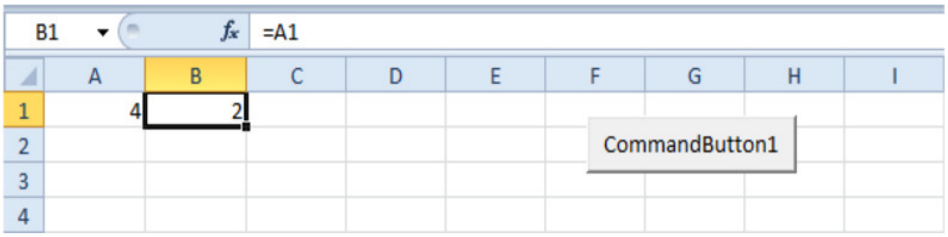
When you click the command button on the worksheet, Excel VBA sets calculation to manual.

Original notes © NFM Noor

2. You can verify this by clicking on File, Options, Formulas.



3. Now when you change the value of cell A1, the value of cell B1 is not recalculated.



You can manually recalculate the workbook by pressing F9.

4. In most situations, you will set calculation to automatic again at the end of your code. Simply add the following code line to achieve this.

```
Application.Calculation = xlCalculationAutomatic
```

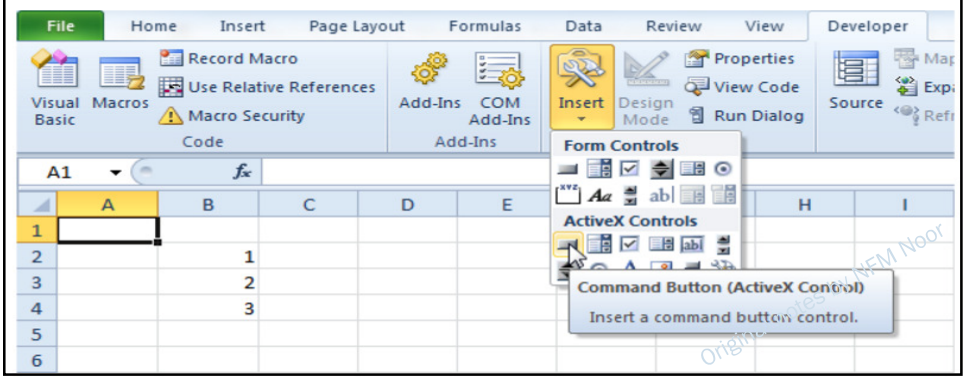
Original notes by NFM Noor

ActiveX Controls

- To create command buttons, text/list boxes etc.

1. On the Developer tab, click Insert.

2. For example, in the ActiveX Controls group, click Command Button to insert a command button control.

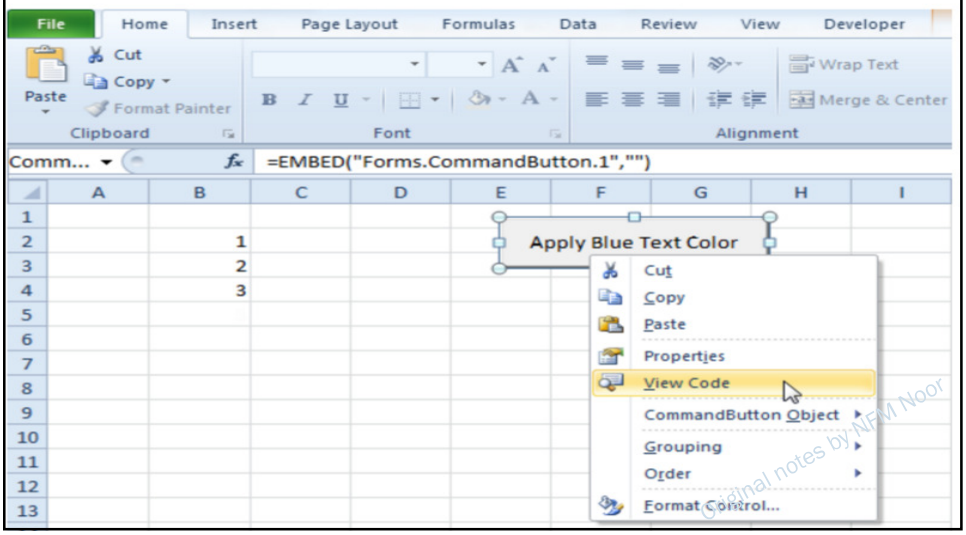


Original notes by NFM Noor

3. Drag a command button on your worksheet.

4. Right click the command button (make sure Design Mode is selected).

5. Click View Code.

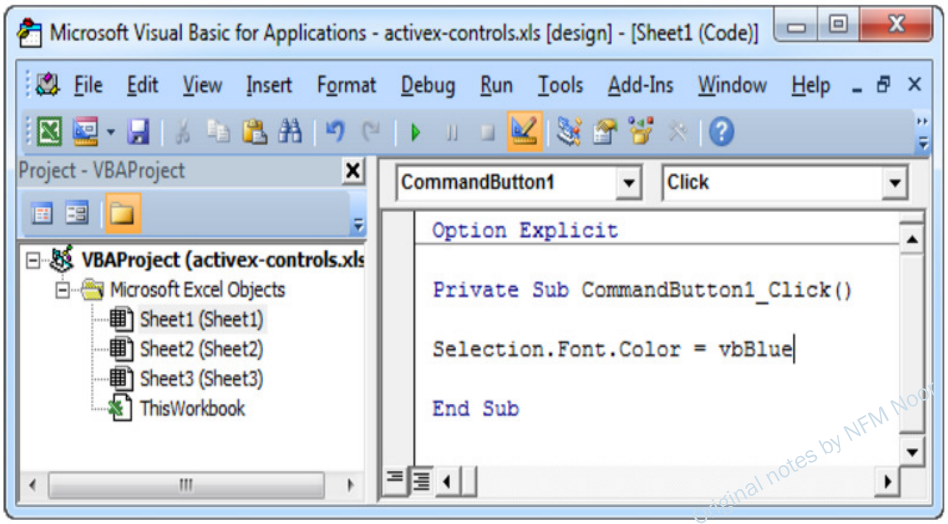


Note: you can change the caption and name of a control by right clicking on the control (make sure Design Mode is selected) and then clicking on Properties. Change the caption of the command button to 'Apply Blue Text Color'. For now, we will leave CommandButton1 as the name of the command button.

The Visual Basic Editor appears.

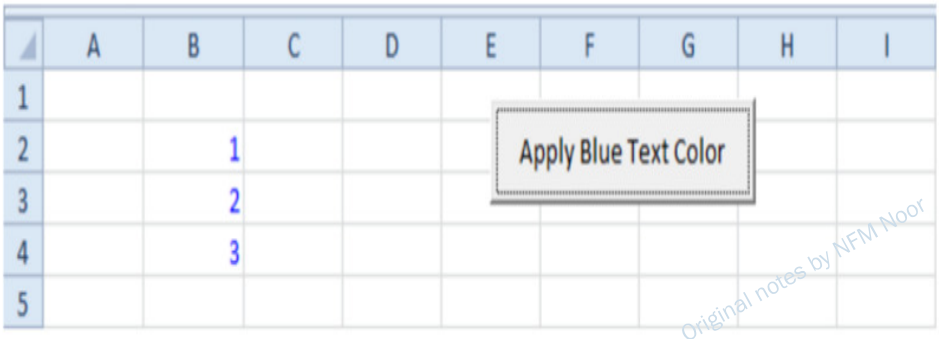


6. Add the code line shown below between Private Sub CommandButton1_Click() and End Sub.



7. Select the range B2:B4 and click the command button (make sure Design Mode is deselected).

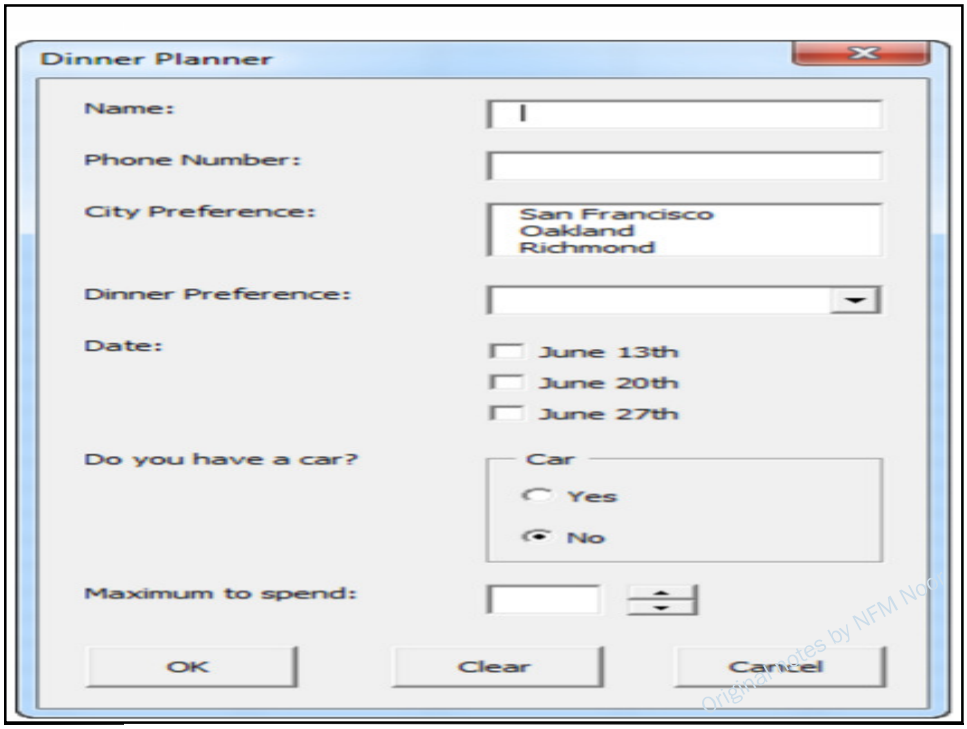
Result:



Userform

- A userform is a custom-built dialog box via VBA.
- Used to acquire information from user to satisfy a VBA procedure.
- Basic controls that can be added to userform includes labels, text/combo/list/check boxes, command/option/spin buttons, frames, images,

Original notes by NFM Noor



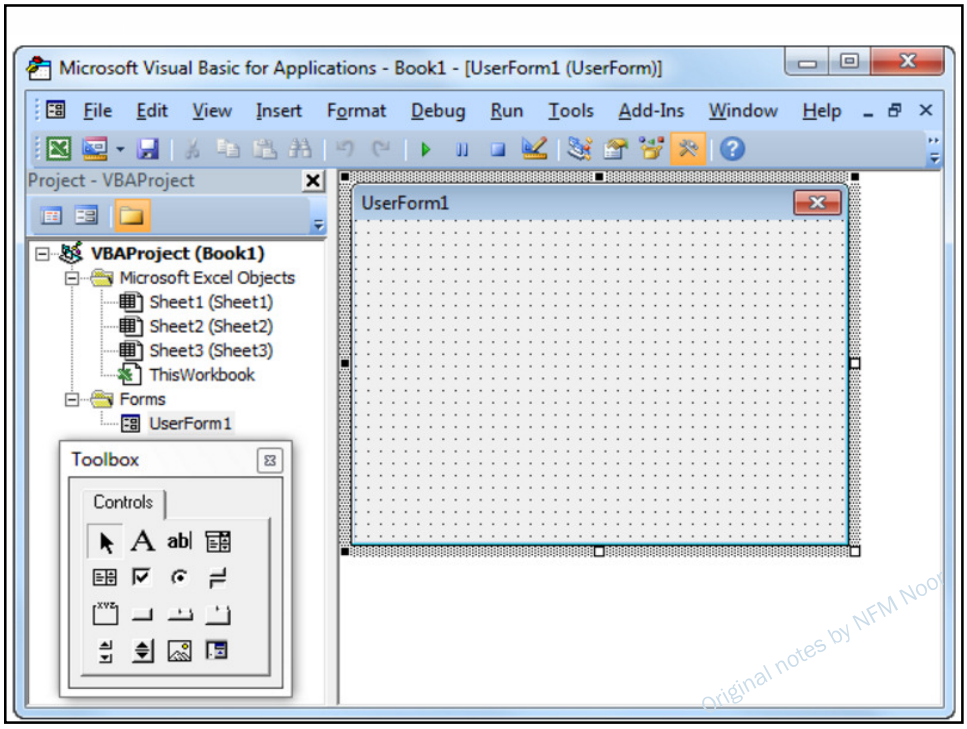
Add the Controls

To add the controls to the Userform, execute the following steps.

1. Open the Visual Basic Editor. If the Project Explorer is not visible, click View, Project Explorer.

2. Click Insert, Userform. If the Toolbox does not appear automatically, click View, Toolbox. Your screen should be set up as below.

Original notes by NFM Noor



3. Add the controls listed in the table below. Once this has been completed, the result should be consistent with the picture of the Userform shown earlier. For example, create a text box control by clicking on TextBox from the Toolbox. Next, you can drag a text box on the Userform. When you arrive at the Car frame, remember to draw this frame first before you place the two option buttons in it.

4. Change the names and captions of the controls according to the table below. Names are used in the Excel VBA code. Captions are those that appear on your screen. It is good practice to change the names of controls. This will make your code easier to read. To change the names and captions of the controls, click View, Properties Window and click on each control.

Original notes by NFM Noor

Control	Name	Caption
Userform	DinnerPlannerUserForm	Dinner Planner
Text Box	NameTextBox	
Text Box	PhoneTextBox	
List Box	CityListBox	
Combo Box	DinnerComboBox	
Check Box	DateCheckBox1	June 13th
Check Box	DateCheckBox2	June 20th
Check Box	DateCheckBox3	June 27th
Frame	CarFrame	Car
Option Button	CarOptionButton1	Yes
Option Button	CarOptionButton2	No
Text Box	MoneyTextBox	
Spin Button	MoneySpinButton	
Command Button	OKButton	OK
Command Button	ClearButton	Clear
Command Button	CancelButton	Cancel
7 Labels	No need to change	Name:, Phone Number:, etc.

Original notes by NFM Noor

Note: a combo box is a drop-down list from where a user can select an item or fill in his/her own choice. Only one of the option buttons can be selected.

Show the Userform

To show the Userform, place a command button on your worksheet and add the following code line:

```
Private Sub CommandButton1_Click()

DinnerPlannerUserForm.Show

End Sub
```

We are now going to create the Sub UserForm_Initialize. When you use the Show method for the Userform, this sub will automatically be executed.

Original notes by NFM Noor

1. Open the Visual Basic Editor.

2. In the Project Explorer, right click on DinnerPlannerUserForm and then click View Code.

3. Choose Userform from the left drop-down list. Choose Initialize from the right drop-down list.

4. Add the following code lines:

Original notes by NFM Noor

```
Private Sub UserForm_Initialize()

'Empty NameTextBox
NameTextBox.Value = ""

'Empty PhoneTextBox
PhoneTextBox.Value = ""

'Empty CityListBox
CityListBox.Clear

'Fill CityListBox
With CityListBox
    .AddItem "San Francisco"
    .AddItem "Oakland"
    .AddItem "Richmond"
End With
```

Original notes by NFM Noor

```
'Empty DinnerComboBox
DinnerComboBox.Clear

'Fill DinnerComboBox
With DinnerComboBox
    .AddItem "Italian"
    .AddItem "Chinese"
    .AddItem "Frites and Meat"
End With

'Uncheck DataCheckBoxes
DateCheckBox1.Value = False
DateCheckBox2.Value = False
DateCheckBox3.Value = False

'Set no car as default
CarOptionButton2.Value = True
```

Original notes by NFM Noor

```
'Empty MoneyTextBox
MoneyTextBox.Value = ""

'Set Focus on NameTextBox
NameTextBox.SetFocus

End Sub
```

Explanation: text boxes are emptied, list boxes and combo boxes are filled, check boxes are unchecked, etc.

Original notes by NFM Noor

Assign the Macros

We have now created the first part of the Userform. Although it looks neat already, nothing will happen yet when we click the command buttons on the Userform.

1. Open the Visual Basic Editor.

2. In the Project Explorer, double click on DinnerPlannerUserForm.

3. Double click on the Money spin button.

4. Add the following code line:

Original notes by NFM Noor

```
Private Sub MoneySpinButton_Change()

MoneyTextBox.Text = MoneySpinButton.Value

End Sub
```

Explanation: this code line updates the text box when you use the spin button.

5. Double click on the OK button.

6. Add the following code lines:

Original notes by NFM Noor

```
Private Sub OKButton_Click()

Dim emptyRow As Long

'Make Sheet1 active
Sheet1.Activate

'Determine emptyRow
emptyRow = WorksheetFunction.CountA(Range("A:A")) + 1

'Transfer information
Cells(emptyRow, 1).Value = NameTextBox.Value
Cells(emptyRow, 2).Value = PhoneTextBox.Value
Cells(emptyRow, 3).Value = CityListBox.Value
Cells(emptyRow, 4).Value = DinnerComboBox.Value
```

Original notes by NFM Noor

```
If DateCheckBox1.Value = True Then Cells(emptyRow, 5).Value =
DateCheckBox1.Caption

If DateCheckBox2.Value = True Then Cells(emptyRow, 5).Value =
Cells(emptyRow, 5).Value & " " & DateCheckBox2.Caption

If DateCheckBox3.Value = True Then Cells(emptyRow, 5).Value =
Cells(emptyRow, 5).Value & " " & DateCheckBox3.Caption

If CarOptionButton1.Value = True Then
    Cells(emptyRow, 6).Value = "Yes"
Else
    Cells(emptyRow, 6).Value = "No"
End If

Cells(emptyRow, 7).Value = MoneyTextBox.Value

End Sub
```

Original notes by NFM Noor

Explanation: first, we activate Sheet1. Next, we determine emptyRow. The variable emptyRow is the first empty row and increases every time a record is added. Finally, we transfer the information from the Userform to the specific columns of emptyRow.

7. Double click on the Clear button.

8. Add the following code line:

```
Private Sub ClearButton_Click()

Call UserForm_Initialize

End Sub
```

Explanation: this code line calls the Sub UserForm_Initialize when you click on the Clear button.

Original notes by NFM Noor

9. Double click on the Cancel Button.

10. Add the following code line:

```
Private Sub CancelButton_Click()

Unload Me

End Sub
```

Explanation: this code line closes the Userform when you click on the Cancel button.

Original notes by NFM Noor

Test the Userform

Exit the Visual Basic Editor, enter the labels shown below into row 1 and test the Userform.

Result:

	A	B	C	D	E	F	G
1	Name	Phone Number	City	Dinner	Date	Car	Maximum to spend
2	Niels	070 540 546	Oakland	Chinese	June 13th	No	30
3	Bregje	070 748 847	San Francisco	Italian	June 13th June 20th	Yes	40
4							
5							

Original notes by NFM Noor

Summary

- A function in Excel VBA can return a value while a sub cannot.
- Objects enable access to other Excel related options: WorksheetFunction, ScreenUpdating, DisplayAlerts and Calculation.
- ActiveXControl allows a user to create various types of commands in VBA using TextBox, ListBox etc.
- A UserForm is a custom-built dialog box, used to acquire information from users to satisfy a VBA procedure.

Original notes by NFM Noor

References

- http://www.excel-easy.com/vba.html
- http://www.excel-vba.com
- http://www.tutorialspoint.com/vba/vba_excel_macros.htm
- https://msdn.microsoft.com/en-us/library/office/ee861528.aspx

Original notes by NFM Noor